

Home Assignment for Software Engineering Team Lead

Assignment Overview:

Create a collaborative GIS application where multiple users can draw and analyze areas on a map in real-time, with the ability to switch between regular map and satellite views.

Core Requirements:

1. Backend:

- Implement WebSocket server for real-time updates
- Create endpoints for:
 - Saving drawn areas with metadata (area name, size calculation)
 - Retrieving all areas within current map bounds
 - User authentication and session management
 - Area versioning and edit history
- Calculate and return the area size in square kilometers for drawn polygons
- Handle concurrent drawing from multiple users

Additional Backend Challenges:

- Implement rate limiting to prevent abuse (max 50 drawing actions per minute per user)
- Add spatial indexing for efficient area queries within map bounds
- Create a caching layer for frequently accessed geographic data
- Implement conflict resolution when multiple users edit the same area simultaneously
- Add data validation for polygon coordinates and prevent invalid shapes
- Log all user actions for audit trail and analytics
- Implement graceful degradation when WebSocket connections fail
- Add health check endpoints for monitoring system status

2. Database Design & Performance:

- Design efficient schema for storing geospatial data
- Implement database migrations
- Add proper indexing for spatial queries
- Handle large datasets (assume 10,000+ polygons per map region)
- Implement soft deletes and data retention policies

3. Security & Reliability:

- Implement input sanitization and validation
- Add CORS configuration for cross-origin requests
- Implement proper error logging and monitoring
- Add request timeout handling
- Secure WebSocket connections with authentication tokens

4. Frontend Map Integration:

- Implement map using Leaflet.js with two base layers:
 - OpenStreetMap for default view
 - Satellite imagery from govmap.gov.il (תצא - תצלום אוויר)
- Create a smooth layer transition when switching between views
- Ensure drawn elements remain properly positioned when switching layers

5. Real-time Drawing Features:

- Allow users to draw polygons on both map types
- Show other users' drawing actions in real-time
- Display area calculations as the polygon is being drawn
- Maintain drawing accuracy when switching between satellite and regular views
- Show active users currently viewing/drawing on the map

6. Technical Challenges:

- Handle coordinate transformations between different map projections
- Manage drawing state across layer switches
- Optimize WebSocket messages for real-time updates
- Calculate areas accurately considering Earth's curvature

Advanced Backend Optimization:

- Implement connection pooling for database access
- Add message queuing for handling high-volume WebSocket traffic
- Implement horizontal scaling considerations (how would you handle multiple server instances?)
- Add telemetry and performance metrics collection

Technical Stack:

- Backend: Your choice of language with WebSocket support
- Database: PostgreSQL with PostGIS extension (or justify alternative choice)
- Frontend: Any modern framework + Leaflet.js
- No third-party plugins for the collaborative features

Focus on:

- Core real-time functionality
- Backend architecture and scalability considerations
- Accurate layer switching
- Basic drawing capabilities
- Essential error handling
- Database design and query optimization

Evaluation Criteria:

- Implementation of real-time features
- Backend architecture and code organization
- Database design and performance considerations
- Security implementation
- Handling of map layer transitions
- Accuracy of area calculations
- Error handling and logging approach
- System scalability and monitoring considerations
- Documentation quality and technical decision explanations

Submission Guidelines:

1. GitHub repository containing:

- Complete source code
 - Database schema and migration files
 - README with:
 - Setup instructions (including database setup)
 - Technical decisions and architecture overview
 - Performance considerations and optimization strategies
 - Security measures implemented
 - Known limitations
 - Future improvements and scaling strategies
 - Testing approach
-

Bonus Points (Optional):

- Docker containerization with docker-compose
- Unit tests for critical backend functionality
- API documentation (OpenAPI/Swagger)
- Performance benchmarking results
- Load testing considerations

This task evaluates understanding of:

- Real-time systems and WebSocket architecture
- Database design and spatial data handling
- System scalability and performance optimization
- Security best practices
- Map projections and coordinates
- GIS concepts
- Concurrent user interactions
- Monitoring and observability
- Software architecture patterns